

Multi-Agent Resource Orchestration Based on D3QN for Network Slicing in 5G Edge-Cloud Networks

Xingqi Wu^{ID}, *Graduate Student Member, IEEE*, Junaid Farooq^{ID}, *Senior Member, IEEE*,
and Juntao Chen^{ID}, *Senior Member, IEEE*



Abstract—Optimizing resource orchestration in network slicing is essential for the performance of diverse applications in 5G edge-cloud networks. This paper introduces a novel approach utilizing multi-agent reinforcement learning (MARL) with a dueling double deep Q-network (D3QN) to efficiently manage dynamic resource provisioning to the different traffic flows. We model a network slicing environment with applications generating stochastic resource demands, simulating real-world virtual network patterns over physical infrastructure. Our MARL-based scheme adapts to the varying needs of traffic flows, balancing compute and memory resource allocation under limited information. Comparative analysis demonstrates the superiority of our approach over traditional static methods, particularly for ultra-reliable low-latency communication (URLLC) traffic flows, by minimizing latency and enhancing resource efficiency. The effectiveness of the proposed framework is validated through extensive simulations, which demonstrate up to 45% higher average utility for URLLC traffic flows and 18% improvement in overall resource efficiency compared with baseline strategies. These results confirm that the framework can simultaneously ensure stringent service requirements and enhance system-wide performance in Next-Generation networks.

Index Terms—5G, network slicing, resource orchestration, MARL, micro-services.

I. INTRODUCTION

NEXT-GENERATION (NextG) networks are rapidly evolving towards being software-defined and virtualized, enabling unprecedented flexibility in managing the physical network operations [1], [2]. A key paradigm in these networks is network slicing (NS), which facilitates the creation of virtualized network segments tailored to the specific needs of diverse applications, including industrial automation, remote healthcare, autonomous vehicles, etc. [3], [4]. Network slices can be created exclusively for a particular type of organization, such as enterprise customers, hospitals, educational

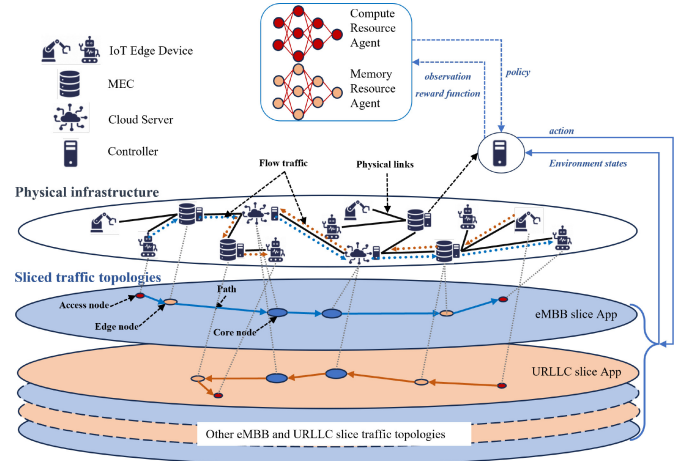


Fig. 1. An abstraction of network slices and traffic flows for applications over an edge-cloud 5G network

institutions, etc. to meet their unique requirements. Thus, the NS paradigm can help service providers to orchestrate the network addressing mission-critical applications requiring real-time control and feedback (e.g., remote driving or telerobotic surgery, which often demands ultra-reliable low-latency communication (URLLC) and enhanced mobile broadband (eMBB) services [5], [6].

In Fig. 1, a representative scenario is presented, which shows how *traffic flows* traverse through fifth generation (5G) network slices across IoT edge devices, multi-access edge computing (MEC) servers, and cloud servers. Each traffic flow has specific requirements for bandwidth, computing power, and memory. The traffic flows generated by users fluctuate in their requirements over time, leading to uncertain and dynamic resource requests. The key objective of NS is to enhance the overall quality of experience (QoE) of applications by satisfying the flow requirements as much as possible. Therefore, an adaptive and efficient resource orchestration mechanism is critically important to meet end-to-end performance requirements of critical traffic flows. For example, end-to-end the latency of traffic flow depends on the bandwidth of the backhaul links, data processing delay at the MEC, and the computing power of core network [7]. Thus, allocating the right combination of available resources along the end-to-end path of the traffic flow is crucial to achieving the latency objectives.

Received 19 April 2025; revised 22 September 2025 and 5 December 2025; accepted 6 December 2025. Date of publication 11 December 2025; date of current version 15 January 2026. The associate editor coordinating the review of this article and approving it for publication was M. Shojafar. (Corresponding author: Xingqi Wu.)

Xingqi Wu and Junaid Farooq are with the Department of Electrical and Computer Engineering, College of Engineering and Computer Science, University of Michigan at Dearborn, Dearborn, MI 48128 USA (e-mail: xingqiwu@umich.edu; mjfarooq@umich.edu).

Juntao Chen is with the Department of Computer and Information Sciences, Fordham University, New York, NY 10023 USA (e-mail: jchen504@fordham.edu).

Digital Object Identifier 10.1109/TNSM.2025.3643340

With the rise of decentralized network architectures, obtaining extensive global network information is constrained by the dispersion of resource nodes and the dynamic nature of the network environment [8]. Traditional static resource allocation [9] methods are inadequate for such dynamic and heterogeneous environments as they fail to capture the intricate and fluctuating traffic dynamics for instantaneous decision-making due to high computational complexity, often leading to sub-optimal resource utilization and an inability to meet service-level agreements (SLAs) at all times [10], [11]. Moreover, these resource allocation frameworks have proved to be less effective in scenarios where resource types, quantities, and locations are highly diverse, particularly unsuited to decentralized networking conditions. Moreover, core nodes often experience a higher volume of traffic because the microservices deployed over the network tend to concurrently overlap on the cloud servers. Therefore, there is a need to actively learn the traffic dynamics in a distributed manner to inform the resource allocation strategy. Apart from these challenges, the network environments are becoming increasingly diverse and expansive. The states of these environments change rapidly on the scale of milliseconds [12], which makes these traditional orchestration less suitable for meeting stringent application requirements. There are growing studies integrating machine learning and reinforcement learning techniques to orchestrate the network. Nevertheless, these learning-based frameworks have usually been applied within centralized network settings without accounting for the potential decentralized topologies and characteristics of edge-cloud systems.

In this paper, the main objective is to provide real-time resource provisioning to meet the dynamic and diverse traffic flow requirements in NextG networks, focusing on compute and memory resources.¹ We first present a dynamic multi-objective optimization problem considering both traffic flow-averaged efficiency and utility metrics. We then propose a resource orchestration framework utilizing multi-agent reinforcement learning (MARL) [13]. In our approach, resource nodes are managed by controllers with two types of learning agents, one is responsible for provisioning compute resource, and the other is for memory resource. These agents can only locally observe aggregated flow patterns and allocated resources from the previous time frame and adapt resource provisioning over time. This can address the challenges in contexts where resource controllers lack visibility into the data of other nodes and the overall network's physical infrastructure and traffic flow topology.

Our proposed framework is designed for practical integration with 5G infrastructures. Each resource agent can be instantiated as a lightweight software component within a software defined networking (SDN) or network function virtualization (NFV) control plane, where it actuates virtualized network functions (VNFs) through Southbound interfaces and consumes monitoring statistics through Northbound interfaces. This preserves responsiveness at the control plane and enables

end-to-end flow actuation at the data plane while relying only on locally observable state, which scales without global synchronization overhead. The design aligns with ongoing standardization on distributed management and orchestration for network slicing and performance measurements in the third generation partnership project (3GPP) [6], [11]. It is also consistent with systems insights on deployment constraints in ad hoc and wireless environments and with slice aware orchestration under SDN and NFV reported in the literature [14], [15], [16].

The proposed MARL-based resource orchestration framework is validated through extensive simulations, demonstrating significant improvements over static and random allocation strategies. Our results highlight the potential of MARL to enhance system utility and efficiency in virtualized networks. The main contributions of this paper are as follows:

- We develop a generalized slice model and introduce traffic flows with stochastic resource demands that represent the virtual communication network patterns over the physical infrastructure network.
- We formulate the flow-based dynamic network resource orchestration problem as a multi objective optimization that aims to maximize the average system utility and overall efficiency.
- We develop a customized resource orchestration scheme utilizing MARL, balancing the varying needs of individual flows, i.e., eMBB and URLLC, amid a highly dynamic decentralized application deployment scenario.
- We provide detailed testing and evaluation of our framework compared to baseline strategies demonstrating significant improvement in resource efficiency without substantial loss in flow performance.

The remainder of this paper is organized as follows: Section II provides the overview of the related works. Section III introduces the slicing and system model. Section IV details the proposed methodology and the MARL-based framework, including the description of actions, observations, reward functions, and the training algorithm. Subsequently, Section V provides the elaboration of the simulation results and a comparative analysis. The paper is concluded in Section VI.

II. RELATED WORKS

There are a variety of works in the literature regarding resource management and orchestration in NextG networks. These can be categorized broadly into three main directions: (i) deployment of static VNFs and microservices over edge-cloud infrastructures, (ii) optimization-based resource provisioning algorithms, and (iii) learning-based resource orchestration schemes.

A. Network Slicing and VNF Chain Deployment

Numerous studies have explored the creation of network slices in distributed edge-cloud infrastructures [17] and the embedding of VNF chains [18], [19], [20], [21]. These works focus primarily on achieving minimizing average latency within computational paths with several constraints such as network architecture, resilience, and robustness. There has been a focus on improving

¹Several other types of resources can also be considered but these do not significantly alter the proposed resource orchestration scheme.

the cost efficiency while implementing NS and deploying VNFs. For instance, [14], [22] have used heuristics to optimally assign VNFs from each network slice with the objectives of reducing the operational cost of network management, such as energy cost, VNF deployment cost, and traffic forwarding cost, etc. Other studies aim to tackle the problem of changing application needs for VNF chains over time [23], [24], [25]. These leverage dynamic programming-based solutions involving heuristics to solve the VNF orchestration problems at scale. Methodologies developed for slice creation and VNF chain placement are applicable only when the information of the functional chains is known prior to the resource allocation stage. However, in actual networks, it can be challenging to implement the frequent and repetitive deployment of network functions in a near-real-time fashion. In this work, our proposed framework is flexible in terms of VNF chain placement since the method can learn the VNF activity and resource needs in an online manner.

B. Optimization-Based Resource Allocation

Once network slices have been defined and VNF chains are deployed, there is a need to optimize the resource (compute, memory, etc.) allocation to VNFs of each slice to satisfy SLAs. Several works have been proposed in this realm to optimize link and node resources to achieve slice performance objectives. For instance, [26], [27] optimize the link resource allocation focusing on maximizing the throughput in order to minimize the communication delay. Studies [28] and [29] present multi-resource allocation problems and have jointly optimized the distinct type of resource such as computational and communication resource using mixed integer linear programming, aiming to maximize the profit of systems and guarantee the satisfaction of response time. Methodologies for reducing energy consumption have been explored by many works as well [30], [31]. Furthermore, an increasing body of research adopts distributed optimization methods to address the scenario where resources are sparsely located [32], [33], [34]. The corresponding solutions based on heuristics are then introduced considering constraints such as resource capacities, resource auctions between the slices and the data centers, and etc. These resource partitioning approaches across the edge-cloud network based on traditional optimization are difficult to implement for their computational complexity and require repeatedly solving the base model, rendering them infeasible in the context of the partially-observable network environment demanding real-time decision-making.

C. Machine Learning-Driven Resource Management

Recent works have increasingly applied learning-based methods for dynamic network resource allocation to address the unique challenges in NextG network. Several studies have utilized the recurring neural network (RNN) and its variant frameworks to perform resource allocation based on prediction of network traffic [35]. These methods have demonstrated their success in predicting resource request patterns. However, the focus is on pattern prediction without considering the network

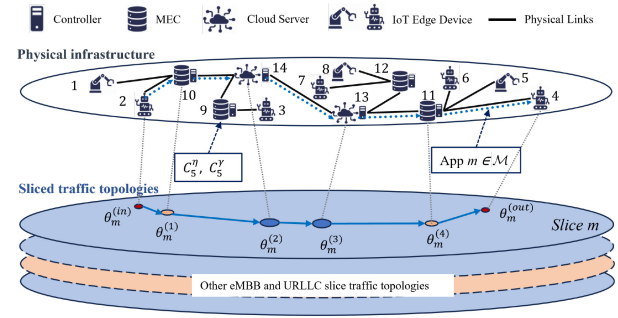


Fig. 2. System Model.

architecture, and the topologies of VNF chains. Learning-based techniques have also been used for dynamic resource allocation decisions [36], [37], [38]. For example, Learning-enhanced stochastic network optimization, incorporating both dual learning techniques and online queue-based control mechanisms has been proposed in [39], [40], which allows the system to learn a safe slicing solution from both historical records and run-time observations. Investigations involving the deep reinforcement learning (DRL) framework, integrating reinforcement learning (RL) algorithms with neural networks (NN) for dynamic resource orchestration, have gained substantial popularity [41], [42], [43], [44], [45]. These resource orchestration frameworks are based on the policies trained through enhancement of traditional RL framework such as by adding target network, refining actions [46], and etc. Furthermore, explorations combining deep Q network (DQN) with various NN architectures have been used for training the orchestration policies [47], [48], incorporating the graph attention network, and generative adversarial network into DQN for training in the face of varying utility to ensure SLA satisfaction and spectrum efficiency.

However, most of these dynamic strategies leveraging online learning are implemented in radio access networks not considering the complexity of the network topology. The articles [40] and [49] have addressed the ability of their learning frameworks utilizing partial observation of network status where the resource nodes are sparsely distributed. Nevertheless, the frameworks rely on a centralized orchestrator for compiling data from distributed units and training management. In this work, we address these limitations by introducing a MARL-based resource orchestration framework designed for decentralized network architectures.

III. SYSTEM MODEL

In this section, we consider a hierarchical 5G edge-cloud infrastructure. We then detail the system model, the characterization of system performance, and the formulation of the problem focused on optimizing system utility and efficiency, as a basis for crafting the resource orchestration policy.

A. Network and Flow Model

We consider a physical network infrastructure consisting of a set of $J_a \in \mathbb{Z}$ access nodes and $J_c \in \mathbb{Z}$ computing nodes (edge and core nodes), denoted by $\mathcal{J} = \{1, 2, \dots, J\}$, and a set of transmission links $\mathcal{L} = \{1, 2, \dots, K\}$, where $J =$

$J_a + J_c$. The topology in Fig. 2 illustrates eight access nodes (1–8), four edge nodes (9–12), and two core nodes (13–14). Each node $j \in \mathcal{J}$ has fixed memory and compute capacities, denoted by C_j^η and C_j^γ , respectively, while each link $k \in \mathcal{L}$ has bandwidth capacity C_k^ζ . Access nodes are assumed to have no compute or memory resources.

A set of M traffic flows, denoted by $\mathcal{M} = \{1, 2, \dots, M\}$, share these resources. To impose a consistent hierarchy throughout the paper, we adopt the following terminology. *Slices* represent logical service classes with distinct QoS requirements. We consider two slice categories, eMBB and URLLC, indexed by $\sigma \in \{1, 2\}$. Each slice σ contains a disjoint subset of traffic flows $\mathcal{M}_\sigma \subset \mathcal{M}$, with $\mathcal{M}_1 \cap \mathcal{M}_2 = \emptyset$. A *traffic flow* is an end-to-end service request belonging to a slice. Each traffic flow consists of a sequence of *microservices*, where each microservice corresponds to one processing stage deployed at a node along the flow path. The microservices associated with URLLC flows exhibit more stringent latency and reliability constraints than those associated with eMBB flows.

Each traffic flow m begins at node $\theta_m^{(in)}$ and terminates at node $\theta_m^{(out)}$, traversing an ordered set of intermediary computing nodes. The flow path is represented by the node set $\Theta_m = \{\theta_m^{(in)}, \theta_m^{(1)}, \theta_m^{(2)}, \dots, \theta_m^{(J_m)}, \theta_m^{(out)}\}$, with J_m intermediary processing stages, and its corresponding link set $\mathcal{L}_m$ connects successive nodes on this path. At each intermediary node $j \in \Theta_m$, a microservice instance is deployed. At time slot t , the resource demand of this microservice is given by $\mathbf{d}_{m,j}^{(t)} = \{\eta_{m,j}^{(t)}, \gamma_{m,j}^{(t)}\}$, representing required memory and compute resources. The bandwidth demand for transmitting data between microservices along link $k \in \mathcal{L}_m$ is denoted by $\zeta_{m,k}^{(t)}$. The corresponding allocated resources are $\mathbf{r}_{m,j}^{(t)} = \{\hat{\eta}_{m,j}^{(t)}, \hat{\gamma}_{m,j}^{(t)}\}$, and $\hat{\zeta}_{m,k}^{(t)}$ respectively.

B. Slice Utility and Efficiency Characterization

The utility of memory resources for a microservice in traffic flow m deployed on node $j \in \Theta_m$ is quantified by the function $f_\sigma(x)$, defined as:

$$f_\sigma(x) = \begin{cases} \frac{1}{1+\exp(-\frac{1}{a_1x+b_1})}, & \sigma = 1, \\ \frac{1}{1+\exp(-\frac{1}{a_2x+b_2})}, & \sigma = 2, \end{cases} \quad (1)$$

where $x \in \mathbb{R}^+$ represents the proportion of the requested memory resources that are satisfied by the allocation. The parameters a_1 and b_1 are selected to ensure that the utility function $f_1(x)$ increases monotonically over the interval $x \in [0, 1]$. This configuration reflects the relaxed and flexible resource requirements of eMBB flows. In contrast, for URLLC flows, the parameters a_2 and b_2 are chosen to align with the stringent requirements of URLLC applications. Specifically, insufficient resource allocation leads to a more pronounced decline in utility for URLLC applications compared to eMBB applications, underscoring the critical importance of adequate resource provisioning to maintain desired performance levels in latency-sensitive use cases. Furthermore, over-provisioning memory resources does not yield additional utility; hence, we define $f_\sigma(x) = 1$ for $x > 1$. Since a traffic flow contains

many microservices or functions, the application-level utility in terms of memory resources, denoted as $u_m^{(t)}$, is computed by aggregating the utilities of all microservices within traffic flow $m \in \mathcal{M}_\sigma$. This can be expressed as:

$$u_m^{(t)} = \frac{1}{|\Theta_m|} \sum_{j \in \Theta_m} f_\sigma \left(\frac{\eta_{m,j}^{(t)}}{\hat{\eta}_{m,j}^{(t)}} \right),$$

where $\eta_{m,j}^{(t)}$ and $\hat{\eta}_{m,j}^{(t)}$ represent the requested and allocated memory resources, respectively, for microservice of traffic flow m deployed on j at time t .

Similarly, we define the utility of compute and bandwidth resources using a monotonically decreasing function $g_\sigma(z)$, described as follows:

$$g_\sigma(z) = \begin{cases} 1 - \frac{1}{1+\exp(-\frac{1}{a_3z+b_3})}, & \sigma = 1, \\ 1 - \frac{1}{1+\exp(-\frac{1}{a_4z+b_4})}, & \sigma = 2, \end{cases} \quad (2)$$

where a_3 and b_3 are chosen to align with the delay performance requirements of eMBB flows, while a_4 and b_4 are selected to meet the more stringent delay requirements of URLLC flows. Therefore, the utility of a traffic flow $m \in \mathcal{M}_\sigma$ in terms of compute and bandwidth resources, denoted as $v_m^{(t)}$ and $w_m^{(t)}$, is computed as follows:

$$w_m^{(t)} = \frac{1}{|\Theta_m|} \sum_{j \in \Theta_m} g_\sigma \left(\tau_{m,j}^c(t) \right),$$

$$v_m^{(t)} = \frac{1}{|\mathcal{L}_m|} \sum_{k \in \mathcal{L}_m} g_\sigma \left(\tau_{m,k}^b(t) \right),$$

where $\tau_{m,j}^c(t)$ and $\tau_{m,k}^b(t)$ denote the compute and transmission latency experienced by microservices on node j and link k , respectively. These latencies are derived from queueing processes under first-come, first-served scheduling. The effective amount of compute and bandwidth resource available to a microservice, accounting for queueing backlogs, is defined as:

$$\gamma_{m,j}^{*(t)} = \min \left\{ \hat{\gamma}_{m,j}^{(t)}, \frac{D_{m,j}^c(t-1)}{T_s} + \gamma_{m,j}^{(t)} \right\}, \quad (3)$$

$$\zeta_{m,k}^{*(t)} = \min \left\{ \min_{k \in \mathcal{L}_m} \hat{\zeta}_{m,k}^{(t)}, \frac{D_{m,k}^b(t-1)}{T_s} + \zeta_{m,k}^{(t)} \right\}. \quad (4)$$

Equations (3) and (4) indicate the update of the amount of compute and bandwidth resources that are actually functional on the microservice of traffic flow m deployed on node j or connected through link k at time t , which signifies that if the resources are over-provisioned, it does not provide any additional benefits on traffic flow m . Conversely, when the resources are allocated insufficiently, the queue of requests starts to develop and hence form the residual block. It is important to note that (4) differs from (3) due to the bottleneck restriction applied to bandwidth resources. Specifically, the bandwidth allocated to each microservice within a traffic flow is constrained by the minimum bandwidth resource allocated to any single microservice in that flow. The variable $D_{m,j}^c(t)$ and $D_{m,k}^b(t)$ indicate the size of the corresponding residual

block, which is set to 0 for $t = 0$, and then it is updated each time using

$$D_{m,j}^c(t) = \max\left\{0, D_{m,j}^c(t) + T_s\left(\gamma_{m,j}^{(t)} - \hat{\gamma}_{m,j}^{(t)}\right)\right\}, \quad (5)$$

$$D_{m,k}^b(t) = \max\left\{0, D_{m,k}^b(t) + T_s\left(\zeta_{m,k}^{(t)} - \min_{k \in \mathcal{L}_m} \hat{\zeta}_{m,k}^{(t)}\right)\right\}. \quad (6)$$

Therefore, the end-to-end (E2E) latency can then be determined using the following proposition.

Proposition 1: The E2E latency at time t , $\tau_m^{(t)}$ can be expressed as follows:

$$\tau_m^{(t)} = \sum_{j \in \Theta_m} \frac{2D_{m,j}^c(t-1) - T_s(\gamma_{m,j}^{*(t)} - \gamma_{m,j}^{(t)})}{2\hat{\gamma}_{m,j}^{(t)}} + \sum_{j \in \Theta_m} \frac{1}{\gamma_{m,j}^{*(t)}} + \sum_{k \in \mathcal{L}_m} \frac{2D_{m,k}^b(t-1) - T_s(\zeta_{m,k}^{*(t)} - \zeta_{m,k}^{(t)})}{\min_{k \in \mathcal{L}_m} \hat{\zeta}_{m,k}^{(t)}}, \quad (7)$$

where $\tau_{m,j}^c(t) = \frac{2D_{m,j}^c(t-1) - T_s(\gamma_{m,j}^{*(t)} - \gamma_{m,j}^{(t)})}{2\hat{\gamma}_{m,j}^{(t)}} + \frac{1}{\gamma_{m,j}^{*(t)}}$ and $\tau_{m,k}^b(t) = \frac{2D_{m,k}^b(t-1) - T_s(\zeta_{m,k}^{*(t)} - \zeta_{m,k}^{(t)})}{\min_{k \in \mathcal{L}_m} \hat{\zeta}_{m,k}^{(t)}}$. The first term represents the E2E compute queueing delay, the second term indicates E2E processing delay, and the third term models the E2E transmission queueing delay respectively.

Proof: See the Appendix. ■

The overall utility of a traffic flow incorporates the utility of the compute resource, memory resource and bandwidth resource. The utility function $U_m(\cdot)$ for traffic flow m at time t is defined as follows:

$$U_m(\mathbf{r}_m^{(t)}) = \alpha_c w_m^{(t)} + \alpha_b v_m^{(t)} + \alpha_\eta u_m^{(t)}, \quad (8)$$

where $\mathbf{r}_m^{(t)} = \{\mathbf{r}_{m,j}^{(t)}, \zeta_{m,k}^{(t)}, \mathbf{d}_{m,j}^{(t)}, \hat{\zeta}_{m,k}^{(t)} : k \in \mathcal{L}_m \text{ and } j \in \Theta_m\}$. The non-negative coefficients α_η , α_c , and α_b are chosen for balancing the constituent terms with $\alpha_\eta = \alpha_c = \alpha_b = \frac{1}{3}$, since we have 3 types of resource considered. Similarly, a traffic flow m in the flow category M_σ at time t has an efficiency of $E_m(\cdot)$, which depends on the utilities of resource provisioning and the amount of resources allocated. The resource efficiency of traffic flow m can be defined as follows:

$$E_m(\mathbf{r}_m^{(t)}) = \frac{\nu_b}{|\mathcal{L}_m|} \sum_{k \in \mathcal{L}_m} \frac{g_\sigma(\tau_{m,k}^b(t)) C_k^\zeta}{\hat{\zeta}_{m,j}^{(t)}} + \frac{\nu_c}{|\Theta_m|} \sum_{j \in \Theta_m} \frac{g_\sigma(\tau_{m,j}^c(t)) C_j^\gamma}{\hat{\gamma}_{m,j}^{(t)}} + \frac{\nu_\eta}{|\Theta_m|} \sum_{j \in \Theta_m} \frac{f_\sigma\left(\frac{\eta_{m,j}^{(t)}}{\hat{\eta}_{m,j}^{(t)}}\right) C_j^\eta}{\hat{\eta}_{m,j}^{(t)}}, \quad (9)$$

where $\nu_\eta = \nu_c = \nu_b = \frac{1}{3}$ are weighting factors chosen equally to weight 3 types of resource efficiencies. In the following subsection, we formalize our problem formulation based on these developed metrics of slice utility and resource efficiency.

C. Energy Consumption Calculation

The energy consumption of computation E_{comp} and transmission E_{trans} per second can be derived as follows:

$$E_{comp}(\cdot) \approx \frac{1}{T_s} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{m \in \mathcal{M}} \sum_{j \in \Theta_m} \Psi_j \hat{\gamma}_{m,j}^{(t)} \tau_{m,j}^c(t), \quad (10)$$

$$E_{trans}(\cdot) \approx \frac{1}{T_s} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{m \in \mathcal{M}} \sum_{k \in \mathcal{L}_m} \left(\frac{P_{tr}}{\kappa} + \varsigma + (H_c + \nu \zeta_{m,k}^{(t)}) \tau_{m,k}^b(t) q \right), \quad (11)$$

In (10), Ψ_j denotes a coefficient (with units of J/bit) determined by the chip architecture at resource node j . In (11), the term $\frac{P_{tr}}{\kappa}$ characterizes the effective transmit power, where P_{tr} is the actual transmission power and $\kappa \in (0, 1]$ denotes the power amplifier efficiency. The parameter ς represents the static circuit power consumption, capturing the constant baseline consumption of essential system hardware. Furthermore, H_c quantifies the per-chain power consumption for each transceiver, while ν is the power consumption per unit bandwidth; thus, $\nu \zeta_{m,k}^{(t)}$ specifies the power required for signal processing. Here, q indicates the number of antennas present at the transmitter.

D. Problem Formulation

The averaged system utility and resource efficiency at time t are computed as the combination of the averaged utility and efficiency of the eMBB and the URLLC flows:

$$\bar{U}(\mathbf{R}_M^{(t)}, t) = \frac{|\mathcal{M}_1|}{|\mathcal{M}|} \bar{U}_1^{(t)}(\mathbf{R}_M^{(t)}) + \frac{|\mathcal{M}_2|}{|\mathcal{M}|} \bar{U}_2^{(t)}(\mathbf{R}_M^{(t)}), \quad (12)$$

$$\bar{E}(\mathbf{R}_M^{(t)}, t) = \frac{|\mathcal{M}_1|}{|\mathcal{M}|} \bar{E}_1^{(t)}(\mathbf{R}_M^{(t)}) + \frac{|\mathcal{M}_2|}{|\mathcal{M}|} \bar{E}_2^{(t)}(\mathbf{R}_M^{(t)}), \quad (13)$$

where

$$\bar{U}_\sigma^{(t)}(\mathbf{R}_M^{(t)}) = \frac{1}{|\mathcal{M}_\sigma|} \sum_{m \in \mathcal{M}_\sigma} U_m(\mathbf{r}_m^{(t)}), \quad \sigma \in \Sigma, \quad (14)$$

$$\bar{E}_\sigma^{(t)}(\mathbf{R}_M^{(t)}) = \frac{1}{|\mathcal{M}_\sigma|} \sum_{m \in \mathcal{M}_\sigma} E_m(\mathbf{r}_m^{(t)}), \quad \sigma \in \Sigma, \quad (15)$$

where $\mathbf{R}_M^{(t)} = \{\mathbf{r}_1^{(t)}, \dots, \mathbf{r}_M^{(t)}\}$. The cardinality of $\mathcal{M}$ and $\mathcal{M}_\sigma$ are represented as $|\mathcal{M}|$ and $|\mathcal{M}_\sigma|$. This measure of system performance encapsulates the relative efficiency and functionality across the respective applications. This formulation can scale to capture a variety of different flows, resource requirements, utility functions, and network topologies.

Finally, the dynamic multi-objective optimization problem can be mathematically formulated as follows:

$$\max_{\hat{\eta}_{m,j}^{(t)}, \hat{\gamma}_{m,j}^{(t)}, \hat{\zeta}_{m,k}^{(t)}} \left(\frac{1}{T} \sum_{t=0}^T \bar{E}(\mathbf{R}_M^{(t)}, t), \frac{1}{T} \sum_{t=0}^T \bar{U}(\mathbf{R}_M^{(t)}, t) \right) \quad (16)$$

$$\text{s.t.} \quad \sum_{m \in \mathcal{M}_j} \hat{\eta}_{m,j}^{(t)} \leq C_j^\eta, \quad \forall j \in \mathcal{J}, \quad (17)$$

$$\sum_{m \in \mathcal{M}_j} \hat{\gamma}_{m,j}^{(t)} \leq C_j^\gamma, \quad \forall j \in \mathcal{J}, \quad (18)$$

$$\sum_{m \in \mathcal{M}^k} \hat{\zeta}_{m,k}^{(t)} \leq C_k^\zeta, \quad \forall k \in \mathcal{L}, \quad (19)$$

where $\mathcal{M}^j$ indicates the set of flows that overlap on the node $j \in \mathcal{J}$ and $\mathcal{M}^k$ indicates the set of flows that cross the link $k \in \mathcal{L}$, respectively. The objective in (16) aims to simultaneously maximize the average system utility and resource efficiency over a time horizon T . The constraints in (17)-(19) ensure resources allocated to the microservices of traffic flows at time t by node j and link k cannot exceed the node's compute and memory resource capacity.

Considering maximizing utility, if there is only one computing node in the network, when $T = 1$, our decision problem can be viewed as a partition problem, i.e., whether the resources within the node can be partitioned into 2 sets to satisfy requests for the eMBB flow and URLLC flow, which means $\bar{U}_1^{(t)}(\mathbf{R}_M^{(t)}) = \bar{U}_2^{(t)}(\mathbf{R}_M^{(t)}) = 1$. Hence, when $T = 1$, our formulated decision problem is reducible to the partition problem. Since the partition problem is a well-known NP-complete problem, our decision problem is also NP-complete, and thus it is NP-hard. We have more than one resource nodes within the network and resource requests of applications change stochastically over time, adding more complexities to the problem. Furthermore, the objectives in the formulation contain non-convex elements. Therefore, given the non-convex, NP-hard, dynamic, and stochastic nature, the problem is difficult to tackle directly.

IV. METHODOLOGY & MARL FRAMEWORK

In this section, we transform the original dynamic multi-objective optimization problem into a multi-agent problem and introduce our solution based on the MARL framework leveraging the dueling double deep Q-network (D3QN), a widely used reinforcement learning algorithm, for agents to learn the optimal policies that maximize expected reward.

A. Q-Learning Preliminaries

We adopt the concept of Q-learning as a model-free approach, circumventing the need for a predictive model by utilizing a Q-function to approximate future reward expectations. The agent selects an action according to the learned policy and receives a reward based on the chosen action and its interaction with environment. The algorithm iteratively refines state-action values to optimize the policy π^* that maximizes the expected return, which is updated according to the Bellman equation:

$$Q^*(s, a) = \mathbb{E} \left[R(s, a) + \lambda \max_{a'} Q^*(s', a') \right], \quad (20)$$

which updates the Q-value until converged. The learning process adjusts the Q-values by blending the old value with the learned value under a learning rate $\mu \in [0, 1]$:

$$Q(s, a) \leftarrow (1 - \mu)Q(s, a) + \mu \left[R(s, a) + \lambda \max_{a'} Q(s', a') \right]. \quad (21)$$

Balancing the exploitation of the learned Q-function with the exploration of decisions is crucial. A prevalent method

for action selection addressing this challenge is the ϵ -greedy strategy, which chooses the action as follows:

$$a = \begin{cases} \arg \max_a Q(s, a) & \text{w.p. } \epsilon, \\ \text{random action from actions space} & \text{w.p. } 1 - \epsilon, \end{cases} \quad (22)$$

where $\epsilon \in [0, 1]$. In the sequel, we adapt the Q-learning framework to a D3QN and extend it to a multi-agent setting with a customized reward function and action space.

B. D3QN Based MARL Architecture

D3QN builds on the DQN framework by incorporating experience replay, dueling network architecture, and a double learning mechanism, all of which contribute to improved stability and performance. Based on the network topology and the D3QN framework, we introduce a systematic learning architecture that assigns independent agents to node controllers responsible for orchestrating the resources. Each specific resource type is managed by a corresponding resource agent in a node controller. We consider two separate agents for each node controlling compute and memory resources. The total number of these agents within the system is thus $2J_c$. Agents for a specific type of resource provisioning may have overlapping information from the environment, and their decisions will jointly contribute to the resource allocation for the traffic flow. Conversely, agents controlling different types of resources do not affect each other. Two hidden fully connected layers combined with the rectifier linear unit (ReLU) as the activation function are used in the agents' NN architecture. The NN outputs a Q value vector indicating the probabilities of taking an action. Fig. 3 depicts the learning architecture for computer and memory resources, which is also applicable to bandwidth resources on the link. The formulations of the action and reward are detailed in the following subsections.

1) *Observations and Actions*: We transform the original optimization problem (16)-(18) presented in Section III into a distributed multi-agent problem with an underlying Markov decision process (MDP) defined by the tuple $(\mathcal{N}, \mathcal{S}, (\mathcal{O}_i)_{i \in \mathcal{N}}, (\mathcal{A}_i)_{i \in \mathcal{N}}, (R_i)_{i \in \mathcal{N}}, P, \gamma)$. The set of all agents is denoted by $\mathcal{N}$ with a cardinality of $|\mathcal{N}| = 2J_c + K$. The total state space of the environment is denoted by $\mathcal{S}$ and can only be partially observed by each agent, which reflects a degree of information degradation. The action space for agent i is given by $\mathcal{A}_i$, where $\mathcal{A}_1 = \mathcal{A}_2 = \dots = \mathcal{A}_{2J_c+K} = \{1, 2, \dots, N_a\}$ with total N_a actions. Matrix P describes the probability of state transitions. The discount factor, λ , captures the decreasing value of future rewards. Furthermore, R_i refers to the reward obtained by agent i following its action, and $\mathcal{O}_i$ denotes the observation of agent i .

Considering a type of resource agent associated with the controller of resource node j , the observation $o_{j,h}^t \in (\mathcal{O}_i)_{i \in \mathcal{N}}$ of the agent at time t can be denoted as

$$o_{j,h}^t = \left[C_{j,h}^t, \sum_{m \in \mathcal{M}_1^j} r_{m,j}^{t-1}, \sum_{m \in \mathcal{M}_2^j} r_{m,j}^{t-1}, \sum_{m \in \mathcal{M}_1^j} \hat{r}_{m,j}^{t-1}, \sum_{m \in \mathcal{M}_2^j} \hat{r}_{m,j}^{t-1}, \sum_{j \in \Theta_m} \sum_{m \in \mathcal{M}_1^j} \hat{r}_{m,j}^{t-1}, \sum_{j \in \Theta_m} \sum_{m \in \mathcal{M}_2^j} \hat{r}_{m,j}^{t-1}, \Lambda_{j,h}^{t-1} \right]. \quad (23)$$

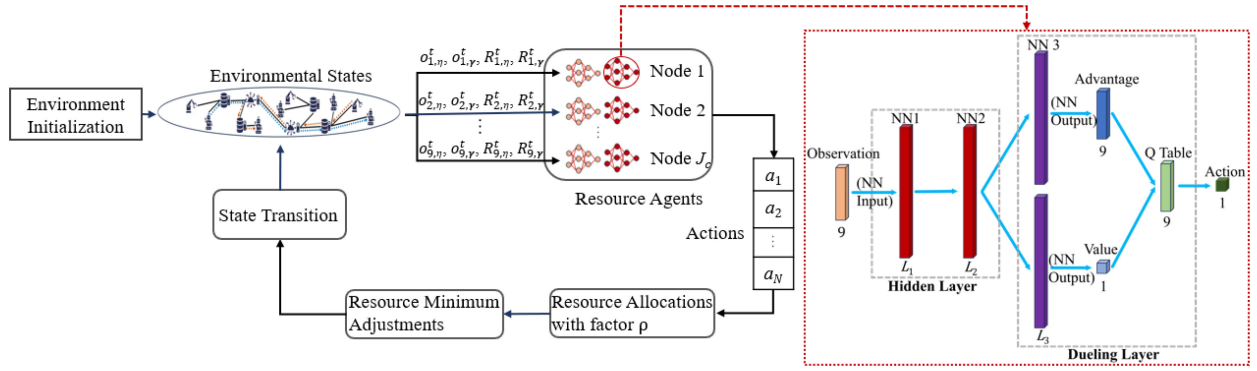


Fig. 3. D3QN based distributed and decentralized MARL architecture for multi-agent dynamic resource orchestration in edge cloud system.

where $h \in \{\eta, \gamma\}$ signifies the index of resource type, while $r_{m,j}^t \in \mathbf{d}_{m,j}^{(t)}$ and $\hat{r}_{m,j}^t \in \mathbf{r}_{m,j}^{(t)}$ represent the requested and allocated resources respectively. Similarly, for resource agent with the controller of the resource link k , the observation $o_{k,\zeta}^t \in (\mathcal{O}_i)_{i \in \mathcal{N}}$ of the agent at time t can be expressed as:

$$o_{k,\zeta}^t = \left[C_k^h, \sum_{m \in \mathcal{M}_1^k} \zeta_{m,k}^{t-1}, \sum_{m \in \mathcal{M}_2^k} \zeta_{m,k}^{t-1}, \sum_{m \in \mathcal{M}_1^k} \hat{\zeta}_{m,k}^{t-1}, \sum_{m \in \mathcal{M}_2^k} \hat{\zeta}_{m,k}^{t-1}, \sum_{m \in \mathcal{M}_1^k} \min\{\hat{\zeta}_{m,k}^{t-1} | k \in \mathcal{L}_m\}, \sum_{m \in \mathcal{M}_2^k} \min\{\hat{\zeta}_{m,k}^{t-1} | k \in \mathcal{L}_m\}, \Lambda_{k,\zeta}^{t-1} \right]. \quad (24)$$

The set of flows in slice σ across the node j and link k is denoted as $\mathcal{M}_\sigma^j$ and $\mathcal{M}_\sigma^k$. The variable $\Lambda_{j,h}^{t-1}$ and $\Lambda_{k,\zeta}^{t-1}$ are arrays that contain actions on the adaptation of resource levels for each slice category. For example $\Lambda_{j,h}^{t-1} = [\rho \cdot \phi_1^{t-1}, \rho \cdot \phi_2^{t-1}]$, suggesting whether to increase, decrease, or maintain resources in current state. The weighting factor $\rho \in \mathbb{R}$ controls the magnitude of resource increment or reduction. Specifically, $\phi_1^t, \phi_2^t \in \{1, -1, 0\}$, and $\phi_\sigma = 1$ indicates increasing the resource for slice σ , while $\phi_\sigma = -1$ and $\phi_\sigma = 0$ suggest decreasing and maintaining the resource. Therefore, considering two slices in the network, there are three actions of resource allocations for the eMBB slice and three actions for the URLLC slice, which makes 9 potential actions for each resource agent. Given an observation, the agent can select an action out of $N_a = 9$ candidate options represented by $a_{j,h}^t$ and $a_{k,\zeta}^t \in \mathcal{A}_i = \{1, 2, \dots, 9\}$. Then, the selected action is mapped to the resource allocation decision ϕ_σ^t through a mapping function $\mathcal{F}(\cdot)$ defined as follows:

$$(\phi_{1,h}^t, \phi_{2,h}^t) = \mathcal{F}(a_{j,h}^t). \quad (25)$$

$$(\phi_{1,\zeta}^t, \phi_{2,\zeta}^t) = \mathcal{F}(a_{k,\zeta}^t). \quad (26)$$

The controller of node j or link k takes $o_{j,h}^{t-1}$ or $o_{k,\zeta}^{t-1}$ as input and returns the action $a_{j,h}^t$ or $a_{k,\zeta}^t$ at time t . Then, the $\hat{r}_{m,j}^t$ and $\hat{\zeta}_{m,k}^t$ can be calculated based on equations (27)-(28):

$$\hat{r}_{m,j}^t = \frac{\sum_{m \in \mathcal{M}_\sigma^j} \hat{r}_{m,j}^{t-1} + \rho \cdot \phi_\sigma^t}{|\mathcal{M}_\sigma^j|}, m \in \mathcal{M}_\sigma. \quad (27)$$

$$\hat{\zeta}_{m,k}^t = \frac{\sum_{m \in \mathcal{M}_\sigma^k} \hat{\zeta}_{m,k}^{t-1} + \rho \cdot \phi_\sigma^t}{|\mathcal{M}_\sigma^k|}, m \in \mathcal{M}_\sigma. \quad (28)$$

We then incorporate the minimum adjustment method based on equations (29)-(30) to prevent resource allocation at node j from exceeding the node capacity. This can be described as follows:

$$\hat{r}_{m,j}^t \leftarrow \hat{r}_{m,j}^t \min \left\{ 1, \frac{C_j^h}{\sum_{m \in \mathcal{M}_1^j \cup \mathcal{M}_2^j} \hat{r}_{m,j}^t} \right\}, \quad (29)$$

$$\hat{\zeta}_{m,k}^t \leftarrow \hat{\zeta}_{m,k}^t \min \left\{ 1, \frac{C_k^\zeta}{\sum_{m \in \mathcal{M}_1^k \cup \mathcal{M}_2^k} \hat{\zeta}_{m,k}^t} \right\}. \quad (30)$$

2) *Reward Function*: The performance of our proposed resource orchestration framework is based on the optimal policies of the agents, which are designed to maximize the system's overall utility and resource efficiency. For both eMBB and URLLC slices, the reward terms depend on the difference between requested and allocated resources, ensuring that over-provisioning is penalized while under-provisioning reduces the realized utility.

Thus, given a resource agent associated with the controller in node j and a traffic flow set $\mathcal{M}_1^j \cup \mathcal{M}_2^j$, the reward at time t is formulated as follows:

$$R_{j,h}^t(o_{j,h}^t, a_{j,h}^t) = -B_1 \sum_{m \in \mathcal{M}_1^j} F_1 \left(\sum_{j' \in \Theta_m} \hat{r}_{m,j'}^t - \sum_{j' \in \Theta_m} r_{m,j'}^t \right) - B_2 \sum_{m \in \mathcal{M}_2^j} F_2 \left(\sum_{j' \in \Theta_m} \hat{r}_{m,j'}^t - \sum_{j' \in \Theta_m} r_{m,j'}^t \right), \quad (31)$$

where B_1, B_2 are positive weighting parameters to balance the contribution of each term within the reward function. The value of B_2 should far exceed B_1 , reflecting the priority of the URLLC slice. As for resource agent associated with the controller in link k , the reward at time t is determined similarly as follows:

$$R_{k,\zeta}^t(o_{k,\zeta}^t, a_{k,\zeta}^t) = -B_3 \sum_{m \in \mathcal{M}_1^k} F_3 \left(\min_{k \in \mathcal{L}_m} (\hat{\zeta}_{m,k}^t) - \max_{k \in \mathcal{L}_m} (\zeta_{m,k}^t) \right) - B_4 \sum_{m \in \mathcal{M}_2^k} F_4 \left(\min_{k \in \mathcal{L}_m} (\hat{\zeta}_{m,k}^t) - \max_{k \in \mathcal{L}_m} (\zeta_{m,k}^t) \right), \quad (32)$$

The function $F_1(\cdot)$, $F_2(\cdot)$, $F_3(\cdot)$, and $F_4(\cdot)$ have the same structure with different coefficients, which is defined based on

the utility function:

$$F_i = \begin{cases} \beta_i \left(\frac{2}{1 + \exp(-q_i(x+x_i))} - 1 \right), & \text{if } e_i < x < 0 \\ -\beta_i/\omega_i, & \text{if } x \geq 0 \text{ or } x \leq e_i \leq 0 \end{cases} \quad (33)$$

where β_i , e_i , q_i , ω_i , and x_i are coefficients which determines the property of function F_i , where $i \in \{1, 2, 3, 4\}$. The reward function is a compound measure that amalgamates two distinct terms: for flows across node j or link k at time t , the former term aggregates the satisfactions in terms of the total amount of a particular type of resource allocated to eMBB traffic flows, and the aggregated resource request of all microservices of the traffic flows at time t . Similarly, the latter term indicates the same process for the flows in the URLLC slice category across node j or link k at time t . Consequently, for the same types of resource agents, the reward function of the resource agent exhibits interdependence between multiple agents.

C. Multi-Agent Resource Orchestration Algorithm

We present the multi-agent resource orchestration algorithm based on the agents' actions, observations, and rewards. The details are presented in Algorithm 1.

It begins by initializing the network nodes, links, topology, resource capacities of nodes, and the application types. At the beginning of each episode, we determine resource request of each traffic flow m , and resource allocation on node j and link k , where $\sum_{m \in \mathcal{M}_1^j} \hat{r}_{m,j}^0 = \sum_{m \in \mathcal{M}_2^j} \hat{r}_{m,j}^0 = \frac{1}{2} C_j^h$ and $\sum_{m \in \mathcal{M}_1^k} \hat{\zeta}_{m,k}^0 = \sum_{m \in \mathcal{M}_2^k} \hat{\zeta}_{m,k}^0 = \frac{1}{2} C_k^\zeta$ signifying an equal split of the resources of each node and link initially. For each time slot t , we select actions based on ϵ -greedy strategy (line 15). Subsequently, resource allocation proceeds based on the selected action (lines 21-22). The decision policies rely on iteratively updating Q networks of the agents based on observations and expected rewards (line 23). If ϵ falls below the defined threshold ϵ_{min} , the algorithm will use the ϵ_{min} in the action selection. After each training episode, we update the requests for microservices.

Agent Coordination and Signaling: To support cooperative behavior among distributed agents with minimal overhead, we implement a lightweight signaling mechanism inspired by decentralized policy sharing schemes. At the beginning of each time slot, each agent broadcasts a compact summary of its local state in the form of a signaling tuple $\xi_j^{(t)} = \{\chi_j^{(t)}, \delta_j^{(t)}\}$, where $\chi_j^{(t)}$ and $\delta_j^{(t)}$ denote current utilization and request levels (compute, memory, and bandwidth). This signaling occurs once per decision interval, and is restricted to immediate neighbors to preserve scalability. The received messages from neighboring agents are aggregated and encoded into the agent's local observation vector, effectively shaping the partial view of the environment used by the reinforcement learning network. The signaling mechanism incurs negligible communication overhead and does not require centralized orchestration.

V. SIMULATION RESULTS AND ANALYSIS

In this section, we first describe our simulation setup and the training process for the proposed algorithm. Then, we

Algorithm 1 Multi-Agent Resource Orchestration Algorithm

Initialization:

- 1: Initialize the set of network nodes $\mathcal{J}$ and links $\mathcal{L}$;
- 2: Adjacent matrix of network topology $\mathbf{A}$;
- 3: The maximum number of traffic flows M ;
- 4: N_s request states and transition matrix $\mathbf{P}$;
- 5: Memory, compute, and bandwidth resource capacities of node j and link k : C_j^η , C_j^γ , and C_k^ζ ;
- 6: Q Networks;
- 7: Set initial exploration rate $\epsilon \leftarrow \epsilon_{max}$;
- 8: Set total episodes and time steps: N_{train} , T ;

Training Phase:

- 9: **for** $n = 1$ to N_{train} **do**
- 10: Set time slot $t = 0$
- 11: Initialize flow traffic, their resource requests of microservices $\mathbf{d}_{m,j}^{(t)}$, $\zeta_{m,k}^{(t)}$;
- 12: Initialize node-wised and link-wised microservice-level resource allocations: $\mathbf{r}_{m,j}^{(t)}$, $\hat{\zeta}_{m,k}^{(t)}$.
- 13: **while** $t < T$ **do**
- 14: Sample random value ϵ_r from Uniform (0,1)
- 15: **if** $\epsilon_r \leq \epsilon$ **then**
- 16: Randomly select action $a_{k,\zeta}^t, \forall k \in \mathcal{L}$ and $a_{j,h}^t, \forall j \in \mathcal{J}, \forall h \in \{\eta, \gamma\}$ from action space.
- 17: **else if** $\epsilon_r > \epsilon$ **then**
- 18: Select action $a_{k,\zeta}^t, \forall k \in \mathcal{L}$ and $a_{j,h}^t, \forall j \in \mathcal{J}, \forall h \in \{\eta, \gamma\}$ based on Q values.
- 19: **end if**
- 20: Map actions to resource allocations $\mathbf{r}_{m,j}^{(t)}$ and $\hat{\zeta}_{m,k}^{(t)}$ using equations (25)-(26) and (27)-(28).
- 21: Adjust resources using equations (29) and (30).
- 22: Calculate corresponding reward based on reward functions $R_{j,h}^t(o_{j,h}^t, a_{j,h}^t)$ and $R_{k,\zeta}^t(o_{k,\zeta}^t, a_{k,\zeta}^t)$ using equations (31) and (32).
- 23: Reduce exploration rate: $\epsilon \leftarrow \max(\epsilon_{min}, \epsilon - \epsilon_\Delta)$
- 24: Update observation $o_{k,\zeta}^t, \forall k \in \mathcal{L}$ and $o_{j,h}^t, \forall j \in \mathcal{J}, \forall h \in \{\eta, \gamma\}$ based on equations (23) and (24).
- 25: Update Q values using equation (21).
- 26: $t \leftarrow t + 1$
- 27: **end while**
- 28: Update $\mathbf{d}_{m,j}^t$ and $\zeta_{m,k}^t$ to the new resource request.
- 29: **end for**

discuss the results and comparative analysis to evaluate the performance of our proposed solution.

A. Simulation Setup

Our simulations are conducted using a Python 3.7 environment, utilizing the Pytorch 1.10.1 library. The network topology used in our simulation experiments is shown in Fig. 4, which is inspired by the edge-cloud architecture in next generation communication applications particularly in industrial settings [50]. The network topology contains three core nodes, each connected to two edge nodes, each having distinct levels of compute and memory resource capacities. Each edge node is interconnected with a multitude of access nodes. Access nodes serve as critical junctures, acting as the

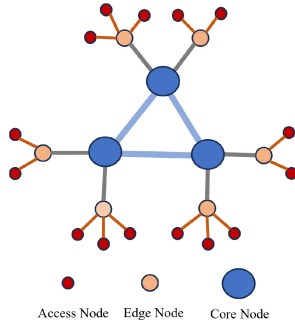


Fig. 4. Example network topology comprising of access nodes, edge nodes, core nodes, and transmission links. The size of the nodes is proportional to the amount of compute and memory resources available at those nodes. The capacity of bandwidth resources are the same across all links.

start and end points for flow traffic. The edge node and the core node are connected through the transmission links with the uniform bandwidth resource.

Note that all experiments in this study are conducted under a fixed topology. In practice, network deployments are subject to dynamic conditions such as node failures, mobility, and scaling events. The proposed framework is still applicable to such environments because each agent operates solely on locally observable states that are updated automatically when connectivity or resources change. Thus, the policy naturally adapts without requiring centralized coordination. Furthermore, transfer learning or policy fine-tuning techniques can be applied so that policies trained in a baseline topology are reused in a perturbed configuration.

During the simulation, we construct traffic flows by initial selection of a pair of access nodes. Subsequently, the intermediate paths are determined by randomly sampling from the available paths between the two nodes. The nodes within any given path of the traffic flow must be unique, thereby excluding the possibility of cyclic paths. During the training phase, the initial resource capacities for both the edge nodes and the core nodes were configured according to normal distributions, $(\mu = 30, \sigma^2 = 1)$ for the edge nodes, $(\mu = 60, \sigma^2 = 2)$ for the core nodes, and $(\mu = 50, \sigma^2 = 1)$ for the links. A comprehensive list of parameter values used in the simulations is provided in Table I. The performance metrics of utility and resource efficiency are computed according to the objective function in (16). Specifically, the average utility is calculated as $\frac{1}{T} \sum_{t=0}^T \bar{U}(\mathbf{R}_M^{(t)}, t)$, and the average resource efficiency as $\frac{1}{T} \sum_{t=0}^T \bar{E}(\mathbf{R}_M^{(t)}, t)$. Furthermore, energy consumption is evaluated based on equations (10) and (11), using E_{comp} for computation energy, and E_{trans} for transmission energy. The energy efficiency per time slot is defined by $\frac{\sum_{t=0}^T \bar{U}(\mathbf{R}_M^{(t)}, t)}{T(E_{\text{comp}} + E_{\text{trans}})T_s}$.

B. Traffic Flow Generation and Model Training

We use a Markov chain (MC) to generate the requirements for each traffic flow $m \in \mathcal{M}_\sigma$. The MC model has N_s states which are randomly sampled within a range specified in Table I. The request ranges for eMBB and URLLC

TABLE I
SIMULATION PARAMETERS

Parameters	Value
Power consumption coefficient, ψ_j	6e-9 J/bit
Transmit Power P_{tr}	1.7 Watts
Circuit characteristic coefficients ($\varsigma, q, H_c, \kappa, \nu$)	20, 0.1, 0.02, 0.4, 5e-10
Number of traffic flows, $\mathcal{M}$	3~12
Compute resource capacity at node j , C_j^c	{30, 60} Gbps
Memory resource capacity at node j , C_j^m	{30, 60} Gb
Bandwidth resource capacity at link k , C_k^ζ	50 GHz
Computation request of eMBB microservices	4.5 ~ 26.5 Mbps
Memory request of eMBB microservices	5 ~ 25 Mb
Bandwidth request of eMBB microservices	3.5 ~ 24.5 MHz
Computation request of URLLC microservices	4.5 ~ 17.5 Mbps
Memory request of URLLC microservices	5 ~ 16 Mb
Bandwidth request of URLLC microservices	3.5 ~ 17.5 MHz
Total training episodes N_{train}	3000
Total testing episodes N_{test}	50
Number of MDP states N_s	10
Learning rate, μ	5e-5
Discount factor, λ	0.99
Time slots for each episode, T	100
Time slot duration, T_s	0.2 s
Function $f_1(\cdot)$ parameters (a_1, b_2)	-40, -30
Function $f_2(\cdot)$ parameters (a_2, b_2)	45, -40
Function $g_1(\cdot)$ parameters (a_3, b_3)	-0.6, -5.4
Function $g_2(\cdot)$ parameters (a_4, b_4)	-3.5, -24.5
Access and computing node number, (J_a, J_c)	15, 9
Transmission link number, K	9
$F_1(\cdot)$ parameters ($\beta_1, q_1, \omega_1, x_1, e_1$)	100, 0.5, 3, 10, -1e4
$F_2(\cdot)$ parameters ($\beta_2, q_2, \omega_2, x_2, e_2$)	100, 3, 3, 1.6, -1e4
$F_3(\cdot)$ parameters ($\beta_3, q_3, \omega_3, x_3, e_3$)	100, 0.5, 1, 10, -5
$F_4(\cdot)$ parameters ($\beta_4, q_4, \omega_4, x_4, e_4$)	100, 3, 1, 1.6, -2.5
Replay buffer size	5e5
Network soft update parameter	5e-3
Resource weighting factor, ρ	3
Reward function weights (B_1, B_2, B_3, B_4)	0.2, 0.8, 0.2, 0.8
Hidden network layer size $[L_1, L_2, L_3]$	[25, 25, 25]
Observation Size	9
Action Size	1
Batch Size	256

microservices are drawn from distinct intervals, reflecting their different service characteristics. eMBB spans broader ranges, while URLLC is limited to narrower ranges aligned with stricter performance demands. The transition matrix $\mathbf{P}$ is constrained such that $\mathbf{P}_{i,j} = 0$ for any $|i - j| > 1$. This implies that state transitions are restricted to neighboring states only based on the given probabilities. Each traffic flow has an independent sampling process from the N_s requirement states. The variability in the number of traffic flows and their respective resource requests can aggregate to exceed network capacity, making selective resource allocation necessary. During the training, the policies of the agents may cause some traffic flows to suffer reduced performance as the system endeavors to maximize overall system performance. An appropriate level of aggregation can help increase the dynamics of the network environment, enabling agents to learn the optimal policy effectively. At the beginning of every episode, we generate a random number of traffic flows, each assigned a static route consisting of a randomly generated node path. Each episode has T time slots. We update resource requests for every episode (every time slot during testing) based on the transition matrix $\mathbf{P}$. As training proceeds, we gradually decrease the exploration to encourage the agent to utilize its learned strategies, preventing it from settling for sub-optimal actions.

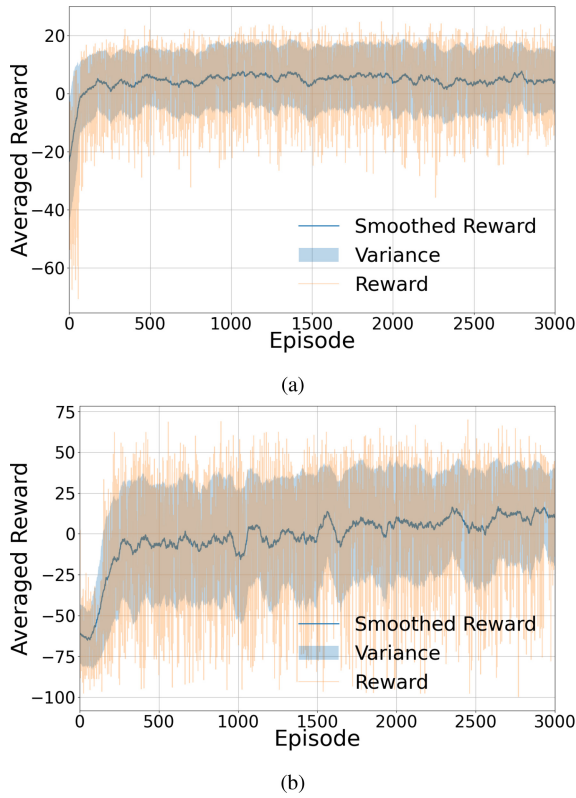


Fig. 5. Averaged training results of compute and memory resource agents (a) and bandwidth agent (b).

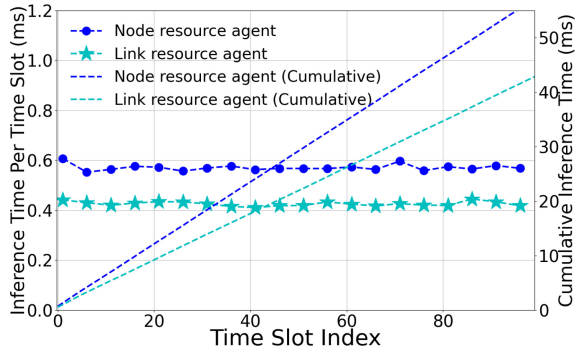


Fig. 6. Average inference time within an episode and cumulative inference time of a resource agent over the testing episodes.

C. Performance Evaluation

The averaged training results for all agents managing compute and bandwidth resources are shown in Fig. 5a and Fig. 5b, demonstrating the convergence trend of the learning process across training episodes. The training pattern for the memory agent is not shown, as it is similar to that of the compute agent, sharing the comparable construct of state and reward functions. We also analyze the inference time for node and link resource agents, as illustrated in Fig. 6. The results show that the average inference time for a resource agent within a time slot is less than 0.5 ms, which is sufficiently low to ensure compliance with the stringent latency requirements of URLLC resource allocation. Next, we use the following baseline methods as benchmarks for comparative analysis.

- 1) *Random Strategy*: Allocation of resources to each slice at every resource node was executed stochastically, wherein a random amount of resources are distributed without regard to the specific needs or priorities of the traffic flows.
- 2) *Static Strategy*: A predetermined static allocation protocol was employed whereby each slice received an equal or proportional. (i.e., invariant allocation of fifty percent of the total resource capacity at every node, regardless of the fluctuating aggregated requirements of the individual flows.)
- 3) *MAD2RO-D3QN*: A multi-agent distributed decentralized resource orchestration framework is implemented and trained using a D3QN network. This baseline approach operates on a static representation of the state space, relying on an estimated average of environment states. We chose to use D3QN to ensure a fair comparison, as traditional A2C is more data-sensitive than DQN and can lead to slow convergence in complex environments, while DQN and DDQN typically require more training steps to achieve convergence compared to D3QN. For each node or link, a resource agent is trained using the same reward functions and training process as in our proposed methods. But this training is based on the average states of a Markov chain for the resource requests of microservices across the node or link. This baseline didn't consider the dynamic nature of the resource request.
- 4) *MAD3RO-DDPG*: A multi-agent distributed decentralized dynamic resource orchestration framework is constructed based on the methodologies outlined in [52], [53], [54]. This framework is trained using a deep deterministic policy gradient, as used in the literature, and employs the same reward function, observations, and training process as our proposed framework. Each node or link has a resource agent that is trained based on the resource requests from eMBB and URLLC microservices with corresponding MCs for state transition. These agents update the actor-critic model parameters at each observation time step with continuous resource allocation decisions. This baseline effectively accounts for the dynamic nature of resource requests.

We note that a centralized orchestration scheme could serve as a theoretical upper bound. However, such a baseline is not included here due to the prohibitive computational complexity of solving the global optimization problem and its limited relevance in realistic 5G edge-cloud deployments, where global state collection and centralized control are impractical. Fig. 7 illustrates how the orchestration strategies align resource allocation with request intensity, and how such alignment governs the resulting latency. In the random and static strategies, there is a persistent mismatch between allocation and demand, either through erratic or unchanging provisioning, resulting in frequent latency degradations. In contrast, the learning-based schemes exhibit progressively better coupling of compute allocation to workload variations. The proposed MAD3RO-D3QN stands out by closely tracking demand shifts with fast upward corrections and judicious resource withdrawal,

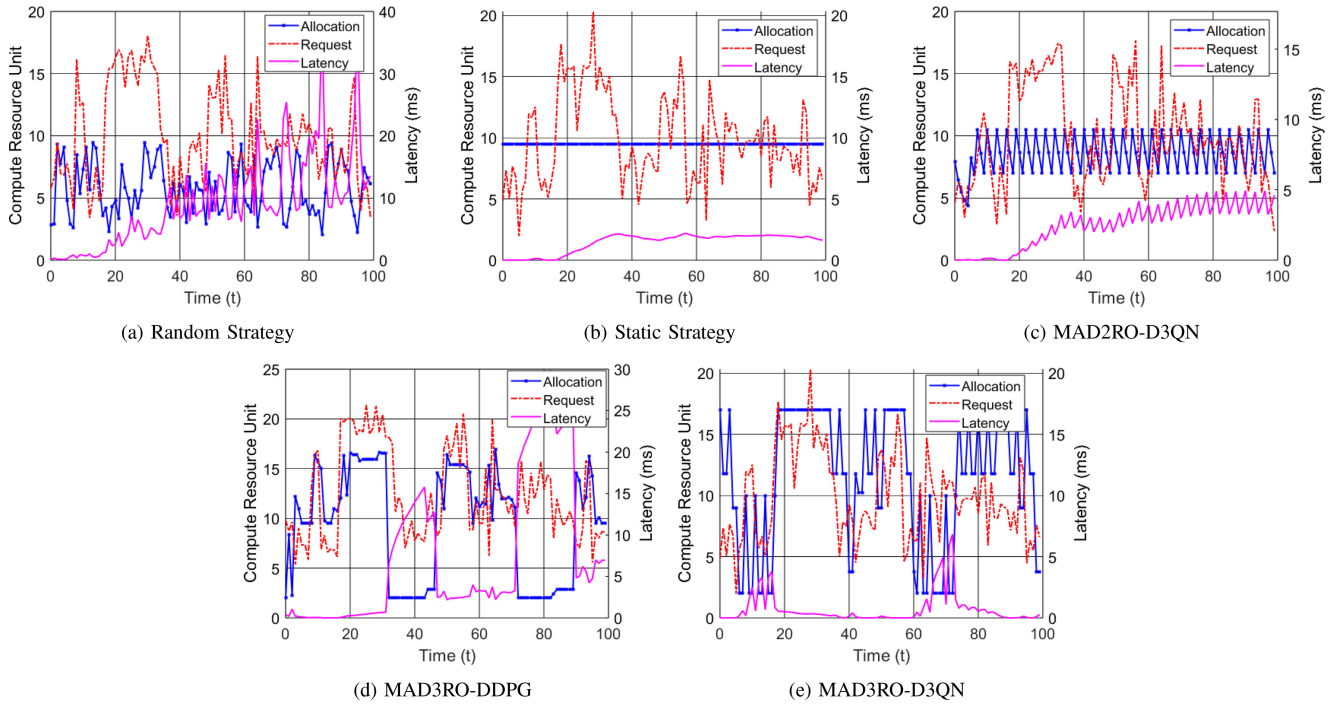


Fig. 7. Simulation results demonstrating compute resource allocation for a micro-service on a resource node varying resource orchestration approaches.

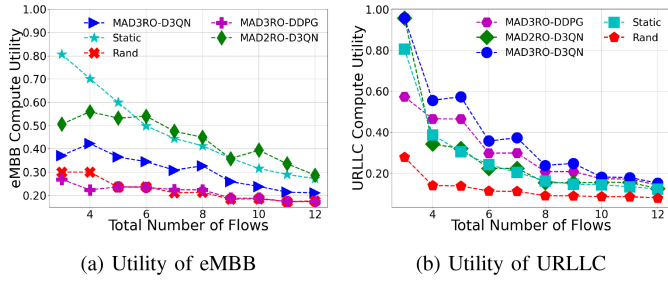


Fig. 8. Averaged compute resource utility of eMBB & URLLC slices vs flow number under different resource orchestration schemes.

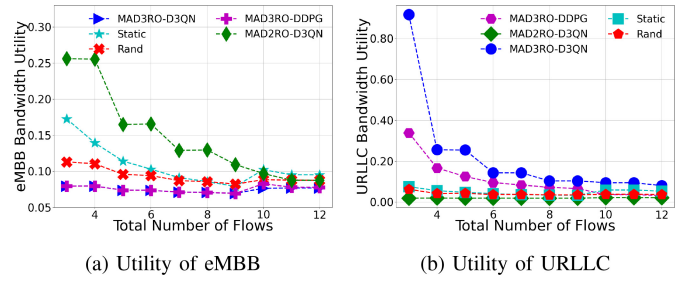


Fig. 10. Averaged bandwidth resource utility of eMBB & URLLC slices vs flow number under different resource orchestration schemes.

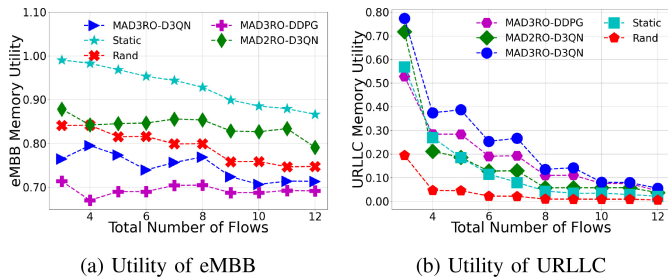


Fig. 9. Averaged memory resource utility of eMBB & URLLC slices vs flow number under different resource orchestration schemes.

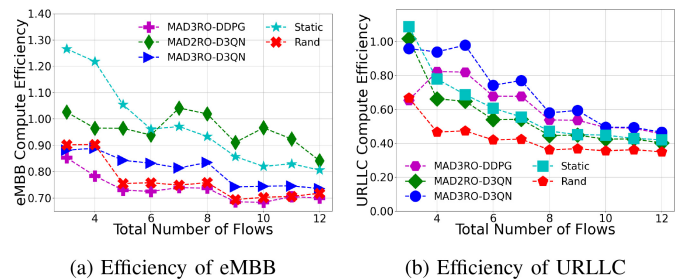


Fig. 11. Averaged compute resource efficiency of eMBB and URLLC slices vs flow number under different resource orchestration schemes.

thereby sustaining near-zero latency throughout the episode. The contrast across subfigures highlights that effective delay control stems not merely from reactivity but from the ability to anticipate and preempt congestion through informed, structure-aware resource decisions.

In the subsequent analysis, we assess the system's utility and efficiency by examining the configuration of compute, memory, and bandwidth resources under highly limited

resource capacity. Specifically, the resource scaling factor for testing is set to 0.25, meaning the compute, memory, and bandwidth capacities are each proportionally reduced to 25% of the levels used during training. The methodology for defining utility and efficiency metrics is presented in Section III. We started the process by varying the total number of traffic flows from 3 to 12 incorporated in the system. We performed N_{test} tests to evaluate performance under a

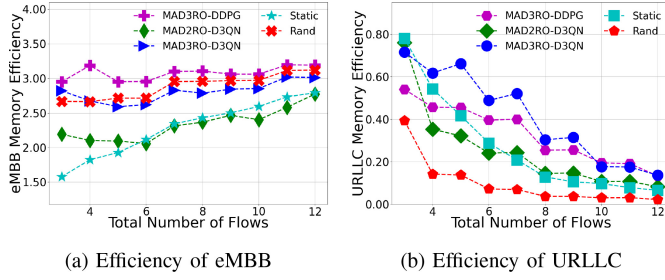


Fig. 12. Averaged memory resource efficiency of eMBB & URLLC slices vs flow number under different resource orchestration schemes.

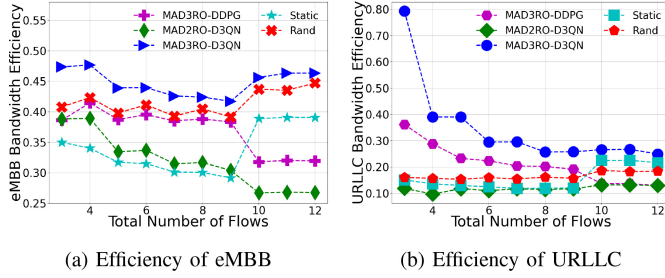


Fig. 13. Averaged bandwidth resource efficiency of eMBB & URLLC slices vs flow number under different resource orchestration schemes.

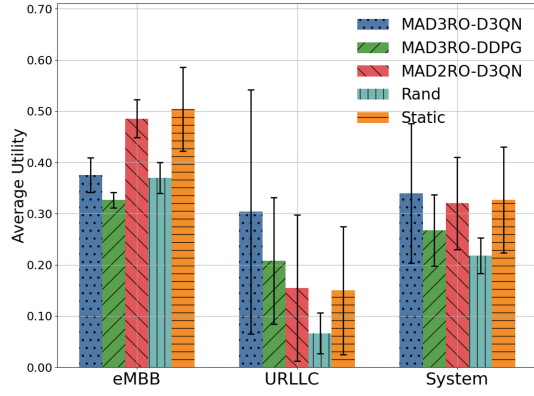


Fig. 14. Average utility of the system and individual slices.

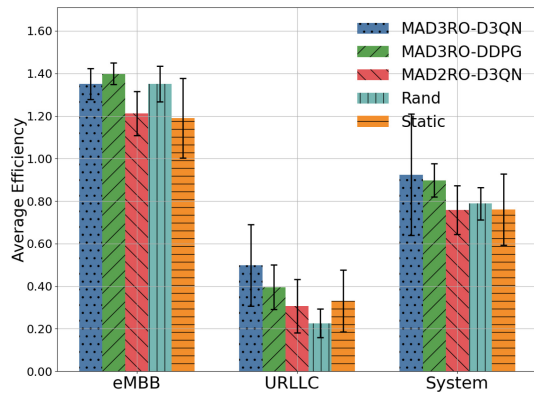


Fig. 15. Average efficiency of the system and individual slices.

fixed $|\mathcal{M}|$. The results obtained from these episodes were then averaged, providing a measure of efficiency and utility under the various orchestration approaches. We further analyze the efficiency of compute, memory, and bandwidth allocation

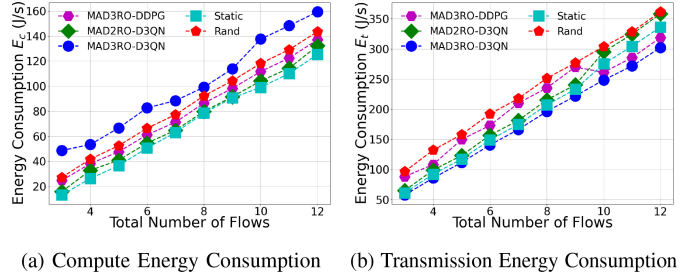


Fig. 16. Averaged compute and transmission energy consumption per second against the number of flows under different resource orchestration schemes.

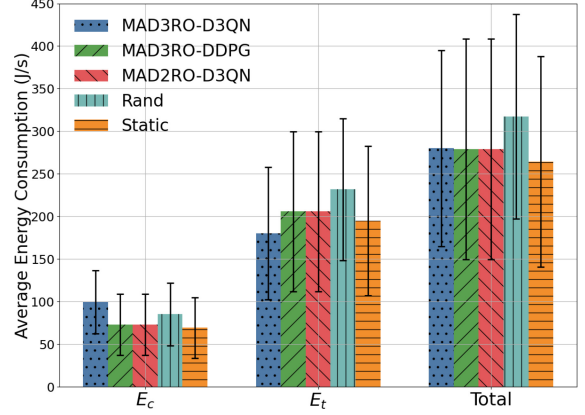


Fig. 17. Average energy consumption of the system.

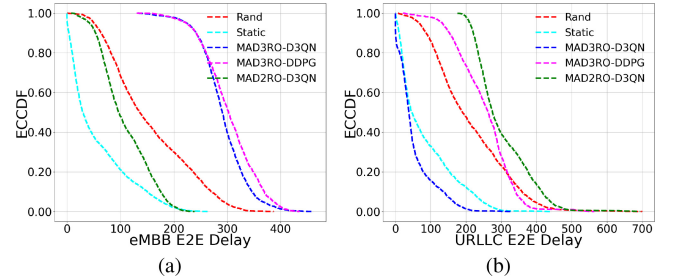


Fig. 18. ECCDF of E2E delay (in ms) of (a) eMBB and (b) URLLC traffic.

under increasing flow count in Fig. 11, Fig. 12, and Fig. 13, respectively. The results consistently demonstrate that the proposed MARL framework sustains high resource utilization efficiency even as the number of concurrent flows increases. Specifically, the advantage becomes more pronounced in high-load scenarios, where decentralized coordination among agents enables adaptive allocation decisions that avoid resource bottlenecks. This confirms the scalability and responsiveness of the framework to dynamic network demands, especially under realistic edge-cloud conditions where flow arrival patterns are heterogeneous and bursty.

Figs. 14 and 15 further summarize the trade-offs across slices. The proposed MAD3RO-D3QN method achieves substantially higher URLLC utility and efficiency while keeping eMBB performance within the range of the other approaches. This reflects a deliberate prioritization structure: under congestion, improving URLLC performance yields disproportionately

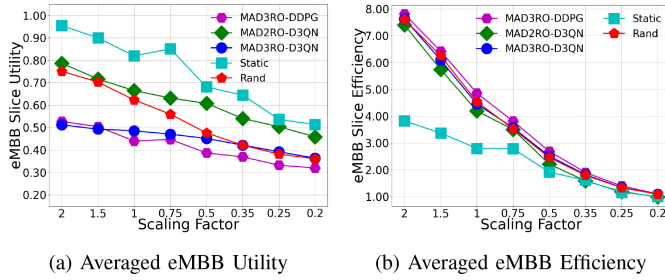


Fig. 19. Average utility and efficiency of eMBB varying node and link resource capacity.

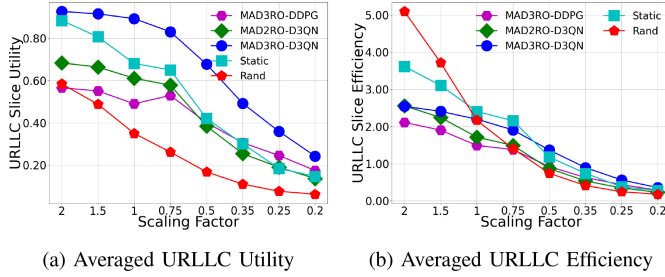


Fig. 20. Average utility and efficiency of URLLC varying node and link resource capacity.

large gains in delay-driven utility, whereas moderate degradation in eMBB utility remains tolerable due to its less stringent latency requirements.

The energy results in Figs. 16 and 17 indicate that the proposed method incurs slightly higher compute energy consumption as the flow count increases. This reflects its faster and more frequent upward adjustments in compute allocation to suppress queue buildup for URLLC flows. However, the same mechanism substantially reduces transmission energy by preventing the long queueing delays that cause repeated retransmissions or prolonged channel occupancy. As a result, the total energy consumption of MAD3RO-D3QN remains comparable to, or lower than, the other learning-based and static strategies. This trade-off is particularly visible in the system-level breakdown of Fig. 17, where lower transmission energy offsets the moderate increase in compute energy.

Fig. 18 presents the empirical complementary cumulative distribution function (ECCDF) of E2E delay across all flows, emphasizing the probability of delay violations under each orchestration strategy. The MAD3RO-D3QN approach exhibits the steepest decay in the tail, indicating a markedly lower probability of high-latency occurrences. This suggests its superior capacity to suppress extreme delays, particularly in latency-sensitive flows. In contrast, the random and static strategies display heavy-tailed distributions, revealing frequent and significant delay outliers. The MAD3RO-DDPG also shows improved tail behavior compared to baseline methods but does not match the sharp decay profile of MAD3RO-D3QN. These findings reinforce the effectiveness of the proposed MARL-based strategy in minimizing delay violations and delivering bounded latency performance in dynamic network conditions.

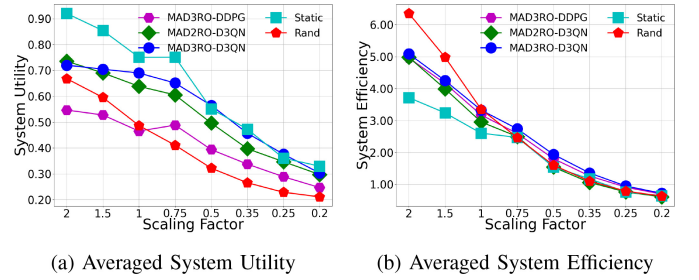


Fig. 21. Average system utility and efficiency varying node and link resource capacity.

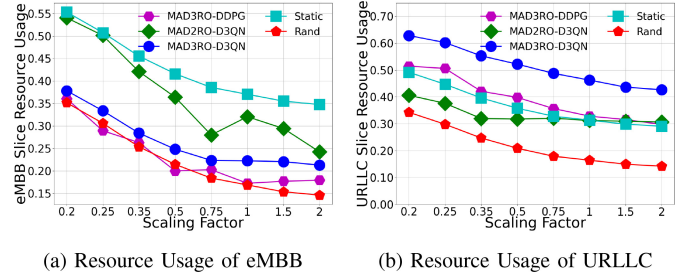


Fig. 22. Average flow-wise resource usage of and utility varying node and link resource capacity.

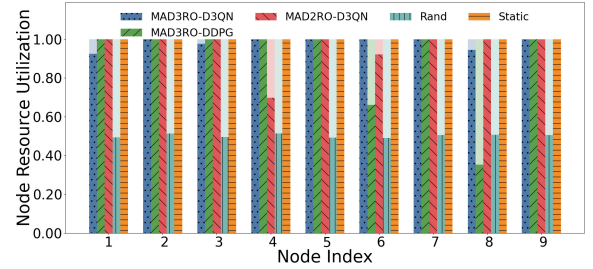


Fig. 23. Comparative analysis of average node resource utilization.

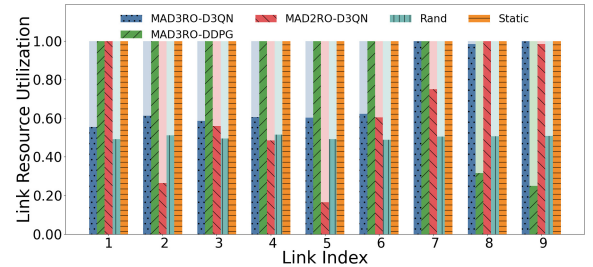


Fig. 24. Comparative analysis of average link resource utilization.

Furthermore, we delved into the utility and efficiency of different slices varying resource capacities at the nodes and links. The illustrations provided in Fig. 19 to Fig. 21 demonstrate these metrics under different orchestration strategies, varying resource capacities. The x-axis represents scaling factors for resource capacity levels, where a scaling factor of 1 corresponds to the baseline resource configuration used during training: 60 compute and memory resource units at the core node, 30 compute and memory resource units at the edge nodes, and 50 bandwidth resource units on the transmission link. As the scaling factor increases or decreases, the compute, memory, and bandwidth capacities are proportionally adjusted

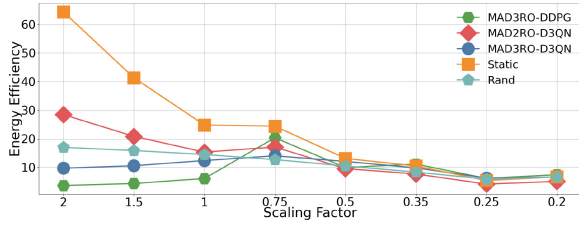


Fig. 25. Average system energy efficiency per time slot varying node and link resource capacity.

relative to their baseline levels. It can be observed from the results that a reduction in the resource capacity of each node correlates with a diminishing trend in both utility and efficiency, regardless of the strategy used. Notably for URLLC slice shown in Fig. 20, in scenarios where resources are abundant, the system utility under MAD3RO-D3QN strategy closely parallels that of the static strategy. However, in environments where resource are more constrained, the proposed MAD3RO-D3QN strategy manifests its superiority by outperforming other strategies compared in terms of both utility and efficiency, which is consistent with system utility and efficiency shown in Fig. 21. This distinction highlights the MAD3RO-D3QN approach's better performance in resource orchestration under resource-limited conditions.

Subsequently, we analyze the average flow-wise resource usage varying resource capacities, as illustrated in Fig. 22. As resource capacities are proportionally increased, the resource usage for both eMBB and URLLC slices tends to converge to a certain level. It is also apparent that both the MAD3RO-DDPG and the proposed MAD3RO-D3QN resource allocation strategies effectively prioritize the URLLC slice by providing it with more resources. Nevertheless, MAD3RO-DDPG generally reserves more resources, resulting in high overall resource efficiency but lower system utility compared to several other resource orchestration schemes. We have also conducted an additional evaluation of node-wise and link-wise resource utilization, as presented in Fig. 23 and Fig. 24. The results demonstrate that the static allocation methods tend to exhaust all available resources regardless of actual demand. In contrast, our proposed framework is able to dynamically reserve a portion of resources, particularly bandwidth on transmission links, by adapting to varying traffic conditions. This resource-aware behavior highlights the greater advantages of our approach in managing network resources efficiently under changing workloads.

Finally, the energy efficiency per time slot is evaluated across various resource capacity scaling factors. As depicted in Fig. 25, the proposed scheme delivers stable energy efficiency throughout all testing scaling scenarios. Although some baseline methods achieve higher energy efficiency under abundant resource conditions, the proposed approach maintains acceptable energy efficiency, particularly in resource-constrained scenarios. The slight reduction in efficiency observed with increased resource availability is primarily due to higher total energy consumption. Overall, the proposed scheme exhibits a competitive energy profile relative to baseline approaches.

VI. CONCLUSION

In this study, we developed a comprehensive network slicing environment model that incorporates application-level traffic characterized by stochastic and heterogeneous resource demands at the microservice level. We proposed a MARL framework for dynamic and distributed resource orchestration, enabling compute and memory provisioning across edge and core nodes in a decentralized manner. The framework is designed to adaptively respond to evolving requirements of traffic flow, such as those of eMBB and URLLC applications, while optimizing both slice utility and system-wide resource efficiency. Our simulation-based evaluations demonstrate that the proposed framework significantly outperforms static, random, and baseline RL-based approaches in terms of utility preservation and efficient use of limited network resources. These findings affirm the promise of MARL as a viable paradigm for scalable and intelligent network management in NextG infrastructures.

APPENDIX

PROOF OF PROPOSITION 1

To derive the equation (7), which determines E2E latency $\tau_m^{(t)}$ of traffic flow m at time t , we proceed as follows: For a microservice of traffic flow m deployed on node j , let T^* be the minimum between the duration of the slot T_s and the time at which the queue empties, i.e.,

$$T^* = \min \left(T_s, \frac{D_{m,j}^c(t-1)}{\hat{\gamma}_{m,j}^{(t)} - \gamma_{m,j}^{(t)}} \right).$$

Moreover, let $\mathcal{D}_{m,j}^c(t, u)$ denote the residual backlog of traffic flow m , u seconds after the beginning of time slot t . Therefore, for any $u \in [0, T^*]$ we have $\mathcal{D}_{m,j}^c(t, u) = D_{m,j}^c(t) - u(\hat{\gamma}_{m,j}^{(t)} - \gamma_{m,j}^{(t)})$, while $\mathcal{D}_{m,j}^c(t, u) = 0$ for $u \in (T^*, T_s]$. Now, the compute queueing delay experienced by the microservice at time $u \in [0, T_s]$ is zero if the queue is empty, and otherwise equal to $\Delta(u, t) = \frac{\mathcal{D}_{m,j}^c(t, u)}{\hat{\gamma}_{m,j}^{(t)}}$. The average delay at time t slot is hence computed as follows:

$$\begin{aligned} \tau_{m,j}^c(t) &= \frac{1}{T_s} \int_0^{T_s} \Delta(u, t) du = \frac{1}{T_s} \int_0^{T^*} \frac{\mathcal{D}_{m,j}^c(t, u)}{\hat{\gamma}_{m,j}^{(t)}} du \\ &= \frac{2T^* D_{m,j}^c(t) - T^{*2} (\hat{\gamma}_{m,j}^{(t)} - \gamma_{m,j}^{(t)})}{2T_s \hat{\gamma}_{m,j}^{(t)}}. \end{aligned}$$

For $T^* = T_s$, we obtain

$$\tau_{m,j}^c(t) = \frac{2D_{m,j}^c(t) - T_s (\hat{\gamma}_{m,j}^{(t)} - \gamma_{m,j}^{(t)})}{2\hat{\gamma}_{m,j}^{(t)}}.$$

And this proof can be applied for transmission queueing process at link k . Since $\tau_m^{(t)} = \sum_{j \in \Theta_m} \tau_{m,j}^c(t) + \sum_{k \in \mathcal{L}_m} \tau_{m,k}^b(t)$, we have equation (7).

REFERENCES

- [1] P. Zhang, H. Yao, and Y. Liu, "Virtual network embedding based on computing, network, and storage resource constraints," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, Oct. 2018.
- [2] T. Taleb, "Toward carrier cloud: Potential, challenges, and solutions," *IEEE Wireless Commun.*, vol. 21, no. 3, pp. 80–91, Jun. 2014.
- [3] P. Rost et al., "Network slicing to enable scalability and flexibility in 5G mobile networks," *IEEE Commun. Mag.*, vol. 55, no. 5, pp. 72–79, May 2017.
- [4] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 2429–2453, 3rd Quart., 2018.
- [5] D. Feng et al., "Toward ultrareliable low-latency communications: Typical scenarios, possible solutions, and open issues," *IEEE Veh. Technol. Mag.*, vol. 14, no. 2, pp. 94–102, Jun. 2019.
- [6] "Study on scenarios and requirements for next generation access technologies, version 14.2.0," 3GPP, Sophia Antipolis, France, Rep. TR 38.913, 2017.
- [7] A. Aijaz, M. Dohler, A. H. Aghvami, V. Friderikos, and M. Frodigh, "Realizing the tactile Internet: Haptic communications over next generation 5G cellular networks," *IEEE Wireless Commun.*, vol. 24, no. 2, pp. 82–89, Apr. 2017.
- [8] M. R. Akdeniz et al., "Millimeter wave channel modeling and cellular capacity evaluation," *IEEE J. Sel. Areas Commun.*, vol. 32, no. 6, pp. 1164–1179, Jun. 2014.
- [9] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. Baruah, J. Gehrke, and C. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proc. 17th IEEE Real-Time Syst. Symp.*, 1996, pp. 288–299.
- [10] K. Samdanis, X. Costa-Perez, and V. Sciancalepore, "From network sharing to multi-tenancy: The 5G network slice broker," *IEEE Commun. Mag.*, vol. 54, no. 7, pp. 32–39, Jul. 2016.
- [11] "Management and orchestration; Concepts, use cases and requirements, version 16.4.0," 3GPP, Sophia Antipolis, France, Rep. TS 28.530, 2020.
- [12] Z. Pi and F. Khan, "An introduction to millimeter-wave mobile broadband systems," *IEEE Commun. Mag.*, vol. 49, no. 6, pp. 101–107, Jun. 2011.
- [13] K. Zhang, Z. Yang, and T. Başar, *Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms*. Cham, Switzerland: Springer Int., 2021, pp. 321–384. [Online]. Available: https://doi.org/10.1007/978-3-030-60990-0_12
- [14] F. Bari, S. R. Chowdhury, R. Ahmed, R. Boutaba, and O. C. M. B. Duarte, "Orchestrating virtualized network functions," *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 4, pp. 725–739, Dec. 2016.
- [15] Y. Wang et al., "Intelligent and collaborative orchestration of network slices," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1239–1253, Mar./Apr. 2023.
- [16] A. Ksentini and P. A. Frangoudis, "Toward slicing-enabled multi-access edge computing in 5G," *IEEE Netw.*, vol. 34, no. 2, pp. 99–105, Mar./Apr. 2020.
- [17] X. Wu, J. Farooq, and J. Chen, "Multi-agent distributed decentralized dynamic resource orchestration in 5G edge-cloud networks," in *Proc. IEEE 13th Int. Conf. Cloud Netw. (CloudNet)*, Rio De Janeiro, Brasil, 2024, pp. 1–8.
- [18] D. Gedia and L. Perigo, "Decision-tree placement algorithm for containerized VoIP VNFs: A network management approach," in *Proc. 25th Conf. Innov. Clouds, Internet Netw. (ICIN)*, Paris, France, 2022, pp. 1–5.
- [19] H. U. Adoga and D. P. Pazaros, "Towards latency-aware vNF placement on heterogeneous hosts at the network edge," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Kuala Lumpur, Malaysia, 2023, pp. 6383–6388.
- [20] X. Wu, J. Farooq, and J. Chen, "Adaptive risk-aware resource orchestration for 5G microservices over multi-tier edge-cloud systems," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Denver, CO, USA, 2024, pp. 359–364.
- [21] D. M. Manias, A. Chouman, J. Naoum-Sawaya, and A. Shami, "Resilient and robust QoS-preserving post-fault VNF placement," *IEEE Netw. Lett.*, vol. 5, no. 4, pp. 270–274, Dec. 2023.
- [22] I. Benkacem, T. Taleb, M. Bagaa, and H. Flinck, "Optimal VNFs placement in CDN slicing over multi-cloud environment," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 616–627, Mar. 2018.
- [23] T. Taleb, M. Bagaa, and A. Ksentini, "User mobility-aware virtual network function placement for virtual 5G network infrastructure," in *Proc. IEEE Int. Conf. Commun. (ICC)*, London, U.K., 2015, pp. 3879–3884.
- [24] X. Cheng, Y. Wu, G. Min, and A. Y. Zomaya, "Network function virtualization in dynamic networks: A stochastic perspective," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2218–2232, Oct. 2018.
- [25] Y. Li, L. T. Xuan Phan, and B. T. Loo, "Network functions virtualization with soft real-time guarantees," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, San Francisco CA, USA, Apr. 2016, pp. 1–9.
- [26] Y. Ma, W. Liang, J. Wu, and Z. Xu, "Throughput maximization of NFV-enabled multicasting in mobile edge cloud networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 2, pp. 393–407, Feb. 2020.
- [27] X. Deng, J. Li, L. Shi, Z. Wei, X. Zhou, and J. Yuan, "Wireless powered mobile edge computing: Dynamic resource allocation and throughput maximization," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2271–2288, Jun. 2022.
- [28] H. Yuan and M. Zhou, "Profit-maximized collaborative computation offloading and resource allocation in distributed cloud and edge computing systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 3, pp. 1277–1287, Jul. 2021.
- [29] C. Gao, A. Shaan, and A. Easwaran, "Deadline-constrained multi-resource task mapping and allocation for edge-cloud systems," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Rio de Janeiro, Brazil, 2022, pp. 5037–5043.
- [30] S. Wang, X. Li, and Y. Gong, "Energy-efficient task offloading and resource allocation for delay-constrained edge-cloud computing networks," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 1, pp. 514–524, Mar. 2024.
- [31] M. A. Hossain and N. Ansari, "Energy aware latency minimization for network slicing enabled edge computing," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 4, pp. 2150–2159, Dec. 2021.
- [32] J. Lee, H. Ko, J. Kim, and S. Pack, "DATA: Dependency-aware task allocation scheme in distributed edge clouds," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7782–7790, Dec. 2020.
- [33] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. M. Leung, "Distributed resource allocation and computation offloading in fog and cloud networks with non-orthogonal multiple access," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12137–12151, Dec. 2018.
- [34] H. Halabian, "Distributed resource allocation optimization in 5G virtualized networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 627–642, Mar. 2019.
- [35] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, E. Dutkiewicz, D. Niyato, and D. I. Kim, "Distributed deep learning at the edge: A novel proactive and cooperative caching framework for mobile edge networks," *IEEE Wireless Commun. Lett.*, vol. 8, no. 4, pp. 1220–1223, Aug. 2019.
- [36] Q. Ye, W. Shi, K. Qu, H. He, W. Zhuang, and X. Shen, "Joint RAN slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 272–288, 2021.
- [37] B. Shang, L. Liu, and Z. Tian, "Deep learning-assisted energy-efficient task offloading in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9619–9624, Sep. 2021.
- [38] D. Ayepah-Mensah, G. Sun, G. O. Boateng, S. Anokye, and G. Liu, "Blockchain-enabled federated learning-based resource allocation and trading for network slicing in 5G," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 654–669, Feb. 2024.
- [39] L. Huang, M. Chen, and Y. Liu, "Learning-aided stochastic network optimization with state prediction," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1810–1820, Aug. 2018.
- [40] X. Cheng, Y. Wu, G. Min, A. Y. Zomaya, and X. Fang, "Safeguard network slicing in 5G: A learning augmented optimization approach," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 7, pp. 1600–1613, Jul. 2020.
- [41] S. Jiang, J. Zheng, F. Yan, and S. Zhao, "Reinforcement-learning-based network slicing and resource allocation for multi-access edge computing networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 10, no. 3, pp. 1132–1145, Jun. 2024.
- [42] Y. Cai, P. Cheng, Z. Chen, M. Ding, B. Vucetic, and Y. Li, "Deep reinforcement learning for online resource allocation in network slicing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7099–7116, Jun. 2024.
- [43] X. Chen et al., "Multi-tenant cross-slice resource orchestration: A deep reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2377–2392, Oct. 2019.
- [44] X. Wu, J. Farooq, and J. Chen, "Joint admission control and resource provisioning for URLLC traffic in O-RAN: A constrained multi-agent reinforcement learning approach," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2025, pp. 1426–1431.

- [45] X. Wu, Y. Wang, J. Farooq, H. Ghazzai, and G. Setti, "SliceWise: Traffic-aware multi-agent RL with temporal encoding for joint slice admission and resource orchestration in O-RAN," *IEEE Netw. Lett.*, early access, Nov. 17, 2025, doi: [10.1109/LNET.2025.3633686](https://doi.org/10.1109/LNET.2025.3633686).
- [46] K. Suh, S. Kim, Y. Ahn, S. Kim, H. Ju, and B. Shim, "Deep reinforcement learning-based network slicing for beyond 5G," *IEEE Access*, vol. 10, pp. 7384–7395, 2022.
- [47] Y. Shao, R. Li, Z. Zhao, and H. Zhang, "Graph attention network-based DRL for network slicing management in dense cellular networks," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Nanjing, China, 2021, pp. 1–6.
- [48] Y. Hua, R. Li, Z. Zhao, X. Chen, and H. Zhang, "GAN-powered deep distributional reinforcement learning for resource management in network slicing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 334–349, Feb. 2020.
- [49] F. Mason, G. Nencioni, and A. Zanella, "Using distributed reinforcement learning for resource orchestration in a network slicing scenario," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 88–102, Feb. 2023.
- [50] T. N. Alrumaih and M. J. Alenazi, "GENIND: An industrial network topology generator," *Alexandria Eng. J.*, vol. 79, pp. 56–71, Sep. 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S111001682300649X>
- [51] M. Al-Saffar and P. Musilek, "Distributed optimization for distribution grids with stochastic DER using multi-agent deep reinforcement learning," *IEEE Access*, vol. 9, pp. 63059–63072, 2021.
- [52] J. Tian, Q. Liu, H. Zhang, and D. Wu, "Multiagent deep-reinforcement-learning-based resource allocation for heterogeneous QoS guarantees for vehicular networks," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 1683–1695, Feb. 2022.
- [53] F. Chai et al., "Multi-agent DDPG based resource allocation in NOMA-enabled satellite IoT," *IEEE Trans. Commun.*, vol. 72, no. 10, pp. 6287–6300, Oct. 2024.
- [54] Y. Xu, H. Zhang, X. Li, F. R. Yu, V. C. Leung, and H. Ji, "Trusted collaboration for MEC-enabled VR video streaming: A multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12167–12180, Sep. 2023.



Xingqi Wu (Graduate Student Member, IEEE) received the B.Eng. degree in electrical engineering from the China University of Mining and Technology-Beijing, Beijing, China, in 2021, and the M.S. degree in electrical and computer engineering from the University of Michigan at Ann Arbor, Ann Arbor, MI, USA, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, University of Michigan at Dearborn. His research interests include network optimization, AI/ML for networks, and edge-cloud computing.



Junaid Farooq (Senior Member, IEEE) received the B.S. degree in electrical engineering from the School of Electrical Engineering and Computer Science, National University of Sciences and Technology, Islamabad, Pakistan, in 2013, the M.S. degree in electrical engineering from the King Abdullah University of Science and Technology, Thuwal, Saudi Arabia, in 2015, and the Ph.D. degree in electrical engineering from the Tandon School of Engineering, New York University, Brooklyn, NY, USA, in 2020. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering, University of Michigan at Dearborn. His research interests include optimization, security, and resilience of communication networks, cyber-physical systems, and the Internet of Things. He was the recipient of the NYU University wide Outstanding Dissertation Award in 2021. Since 2025, he has been serving as an Associate Editor for the IEEE TRANSACTIONS ON COMMUNICATIONS.



Juntao Chen (Senior Member, IEEE) received the B.Eng. degree (with Hons.) in electrical engineering and automation from Central South University, Changsha, China, in 2014, and the Ph.D. degree in electrical engineering from New York University (NYU), Brooklyn, NY, USA, in 2020. He is currently an Assistant Professor with the Department of Computer and Information Sciences and an Affiliated Faculty Member with the Fordham Center of Cybersecurity, Fordham University, New York, USA. His research interests include cyber-physical security and resilience, game and decision theory, network optimization and learning, artificial intelligence, and equitable smart cities. He was a recipient of the Ernst Weber Fellowship, the Dante Youla Award, and the Alexander Hessel Award for the Best Ph.D. Dissertation in electrical engineering from NYU.